

CHƯƠNG 1. GIỚI THIỆU

1.1. Tổng quan:

Nội dung của luận văn chủ yếu tập trung vào thiết kế một digital NPU vừa đủ đơn giản ổn định và tiết kiệm năng lượng để hỗ trợ các ứng dụng biên đơn giản như keyword-spotting và ECG.

1.2. Tình hình nghiên cứu trong - ngoài nước và lý do chọn đề tài:

Các nghiên cứu gần đây cho thấy các thuật toán Học máy (ML) mang lại lợi ích rất lớn, đặc biệt là trong mạng lưới IoT và các thiết bị biên công suất thấp. Thay vì phải gửi dữ liệu thu thập từ các cảm biến hay cơ cấu chấp hành về trung tâm dữ liệu xử lý, thiết bị biên có thể thực hiện toàn bộ quá trình tính toán ngay tại chỗ. Dù gặp nhiều rào cản do hạn chế về năng lượng và dung lượng bộ nhớ, nhu cầu triển khai các thuật toán Học sâu (DL) trực tiếp trên thiết bị biên vẫn tăng mạnh nhằm giảm thiểu độ trễ và bảo vệ quyền riêng tư dữ liệu.

Để giải quyết các thách thức này, nhiều nghiên cứu đã tập trung thiết kế các bộ tăng tốc phần cứng hiệu quả. Vì bản chất của ML/DL chứa rất nhiều phép tính lặp đi lặp lại, việc sử dụng một khối phần cứng chuyên dụng thường được gọi là Bộ tăng tốc học sâu (DLA), Đơn vị xử lý thần kinh (NPU) hoặc Đơn vị xử lý AI (APU) sẽ giúp xử lý các phép toán này nhanh hơn và tiết kiệm năng lượng hơn. Khả năng tối ưu của DLA dựa trên ba yếu tố cốt lõi: khả năng tái sử dụng dữ liệu, mô hình tính toán cố định và tốc độ truy xuất bộ nhớ nhanh. Chính vì lý do đó, xu hướng thiết kế bộ tăng tốc DL hiện nay đang dịch chuyển mạnh mẽ sang kiến trúc phi Von Neumann, giúp tăng cường khả năng tính toán song song và tối ưu hóa hiệu suất sử dụng của các Phần tử xử lý (PE).

Một trong những giải pháp rất thường thấy hiện nay là tích hợp chức năng tính toán vào chính bộ nhớ để giảm tối đa năng lượng hao phí và độ trễ trên đường truyền dữ liệu (CIM). Điều này làm giảm đáng kể chi phí tính toán so với kiến trúc Von Neumann truyền thống. Tuy nhiên CIM vẫn là một công nghệ mới và tiềm ẩn rất nhiều rủi ro trong quá trình tape out, do đó nhu cầu về một khối digital MAC array đi kèm đóng vai trò “fallback” là thiết yếu.

1.3. Nhiệm vụ đề tài:

Thiết kế mảng MAC array INT8 sử dụng chung dataflow với một IP CIM hiện có đóng vai trò fallback.

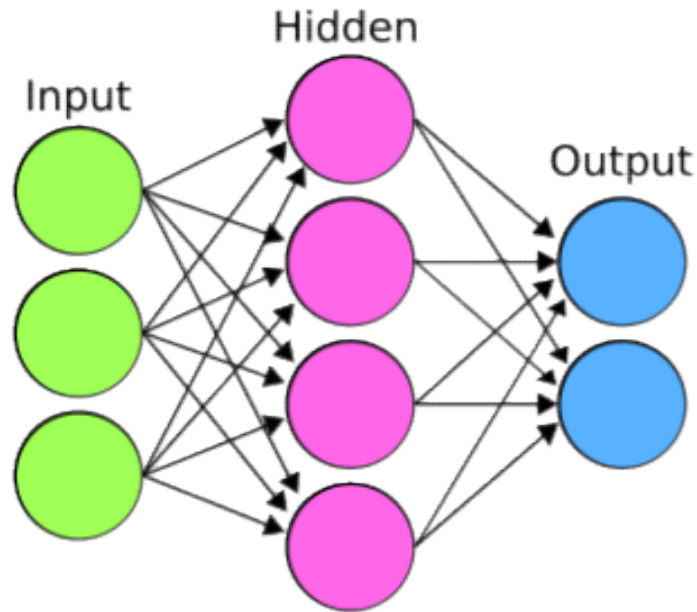
CHƯƠNG 2. BỐI CẢNH & ĐỘNG LỰC

Chương này tập trung trình bày về một số khái niệm cơ bản cần thiết cho việc hiểu và thực hiện đề tài cũng như cho ta thấy được cái nhìn tổng quan về nhu cầu hiện có dành cho các bộ tăng tốc AI phần cứng (hardware AI accelerators) cùng các điểm hạn chế của các nghiên cứu hiện có, qua đó đề xuất hướng tiếp cận mới hơn.

2.1. Cơ sở toán học mạng Neural – Phép nhân tích chập:

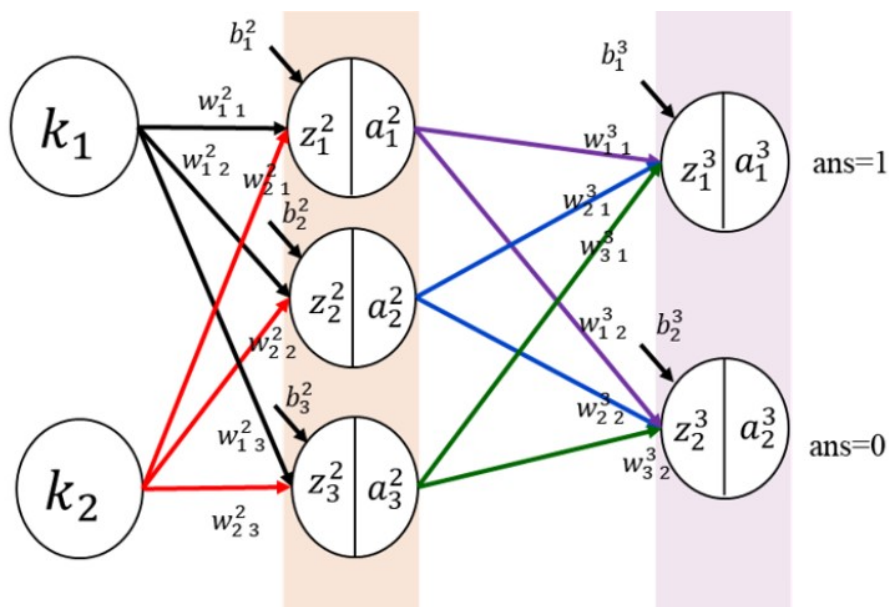
Artificial Neural Network là thuật ngữ chung chỉ các mô hình mạng neuron nhân tạo. Qua quá trình phát triển, nhiều kiến trúc ANN khác nhau đã được đề xuất và công bố. Trong phạm vi phần này, nguyên lý hoạt động được trình bày dựa trên mô hình tổng quát của ANN theo tài liệu của tổ chức LSI Design Contest [1].

Não người chứa một số lượng lớn neuron sinh học, có chức năng tiếp nhận tín hiệu đầu vào và tạo ra tín hiệu đầu ra tương ứng. Ví dụ, khi tay tiếp xúc với một vật có nhiệt độ cao, tín hiệu nhiệt được truyền đến tủy sống thông qua hệ thống tế bào thần kinh, từ đó tạo ra phản xạ rút tay. ANN là mô hình toán học mô phỏng nguyên lý hoạt động của hệ thần kinh sinh học, với cấu trúc cơ bản gồm ba lớp: (1) lớp đầu vào (input layer), (2) lớp ẩn (hidden layer) và (3) lớp đầu ra (output layer), được minh họa trên Hình 1.1. Cấu trúc này được gọi là cấu trúc ba lớp. Đối với các mạng có độ phức tạp cao hơn, số lượng lớp ẩn có thể lớn hơn một [1].



Hình 2.1: ba lớp cơ bản của 1 mạng neural

Tín hiệu được đưa vào tại lớp (1) và truyền ra tại lớp (3). Trước khi đưa tín hiệu vào mạng, tại đầu ra tồn tại một giá trị giám sát (supervisor), tương ứng với đầu ra kỳ vọng của mạng. Mục tiêu huấn luyện là điều chỉnh các tham số của mạng sao cho sai lệch giữa đầu ra thực tế (output) và giá trị giám sát là nhỏ nhất [1].



Hình 2.2: mạng neural với các trọng số

Hình 1.2 minh họa một mạng neuron theo cấu trúc ba lớp cùng các tham số tương ứng với từng lớp. Ý nghĩa của các tham số được định nghĩa như sau:

k_i là tín hiệu vào tại lớp đầu vào.

w_{ij}^2 là trọng số (weight) liên kết từ lớp đầu vào đến lớp ẩn.

b_i^2 là hệ số điều chỉnh (bias) của lớp ẩn.

z_i^2 là tín hiệu đầu vào của lớp ẩn.

a_i^2 là tín hiệu đầu ra của lớp ẩn.

w_{ij}^3 là trọng số (weight) liên kết từ lớp ẩn đến lớp đầu ra.

b_i^3 là hệ số điều chỉnh (bias) của lớp đầu ra.

z_i^3 là tín hiệu đầu vào của lớp đầu ra.

a_i^3 là tín hiệu đầu ra của lớp đầu ra.

t_i là giá trị giám sát (supervisor), với $t_1 = 1$ và $t_2 = 0$.

Mối quan hệ giữa các tham số nêu trên được biểu diễn theo hai chiều: chiều thuận (forward) và chiều nghịch (backward). Theo chiều thuận, tín hiệu đầu vào tại lớp đầu vào (k_i), cùng các trọng số và hệ số điều chỉnh (w_{ij}^2 , b_i^2 , w_{ij}^3 , b_i^3) được xem là các đại lượng đã biết; mục tiêu là xác định đầu ra a_i^3 của lớp đầu ra theo các công thức (1.1), (1.2), (1.3) và (1.4).

$$\begin{bmatrix} z_1^2 \\ z_2^2 \\ z_3^2 \end{bmatrix} = \begin{bmatrix} w_{11}^2 & w_{12}^2 \\ w_{21}^2 & w_{22}^2 \\ w_{31}^2 & w_{32}^2 \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} + \begin{bmatrix} b_1^2 \\ b_2^2 \\ b_3^2 \end{bmatrix} \quad (1.1)$$

$$\begin{bmatrix} a_1^2 \\ a_2^2 \\ a_3^2 \end{bmatrix} = \begin{bmatrix} a(z_1^2) \\ a(z_2^2) \\ a(z_3^2) \end{bmatrix} \quad (1.2)$$

$$\begin{bmatrix} z_1^3 \\ z_2^3 \end{bmatrix} = \begin{bmatrix} w_{11}^3 & w_{21}^3 & w_{31}^3 \\ w_{12}^3 & w_{22}^3 & w_{32}^3 \end{bmatrix} \begin{bmatrix} a_1^2 \\ a_2^2 \\ a_3^2 \end{bmatrix} + \begin{bmatrix} b_1^3 \\ b_2^3 \end{bmatrix} \quad (1.3)$$

$$\begin{bmatrix} a_1^3 \\ a_2^3 \end{bmatrix} = \begin{bmatrix} a(z_1^3) \\ a(z_2^3) \end{bmatrix} \quad (1.4)$$

Trong các công thức (1.2) và (1.4), hàm kích hoạt (activation function) có thể là bất kỳ hàm khả vi và có tính chuẩn hóa nào [1]. Chẳng hạn, khi sử dụng hàm tansig, công thức (1.2) được viết lại thành công thức (1.5).

$$a_i^2 = \text{tansig}(z_i^2) = \frac{2}{1+e^{-2z_i^2}} - 1, i = 1,2,3 \quad (1.5)$$

Đặt:

$$K = \begin{bmatrix} k_1 \\ k_2 \end{bmatrix}$$

$$W2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 \\ w_{21}^2 & w_{22}^2 \\ w_{13}^2 & w_{23}^2 \end{bmatrix}$$

$$B2 = \begin{bmatrix} b_1^2 \\ b_2^2 \\ b_3^2 \end{bmatrix}$$

$$Z2 = \begin{bmatrix} z_1^2 \\ z_2^2 \\ z_3^2 \end{bmatrix}$$

$$A2 = \begin{bmatrix} a_1^2 \\ a_2^2 \\ a_3^2 \end{bmatrix}$$

$$W3 = \begin{bmatrix} w_{11}^3 & w_{21}^3 & w_{31}^3 \\ w_{12}^3 & w_{22}^3 & w_{32}^3 \end{bmatrix}$$

$$B3 = \begin{bmatrix} b_1^3 \\ b_2^3 \end{bmatrix}$$

$$Z3 = \begin{bmatrix} z_1^3 \\ z_2^3 \end{bmatrix}$$

$$A3 = \begin{bmatrix} a_1^3 \\ a_2^3 \end{bmatrix}$$

Khi đó, các công thức (1.1)–(1.4) được viết lại dưới dạng ma trận như sau:

$$Z_2 = W_2 \cdot K + B_2 \quad (1.6)$$

$$A_2 = A(Z_2) \quad (1.7)$$

$$Z_3 = W_3 \cdot A_2 + B_3 \quad (1.8)$$

$$A_3 = A(Z_3) \quad (1.9)$$

Với mỗi vector đầu vào K thứ n , giá trị hàm lỗi bình phương (square error function) tương ứng được xác định theo công thức (1.10):

$$C_n = \frac{1}{2} \left\{ (a_1^3[n] - t_1[n])^2 + (a_2^3[n] - t_2[n])^2 \right\} \quad (1.10)$$

Khi đưa toàn bộ n vector vào lớp đầu vào, hàm giá (cost function) C được xác định bằng tổng các hàm lỗi bình phương tương ứng, theo công thức (1.11), trong đó C_n được tính theo công thức (1.10).

$$C = C_1 + C_2 + \dots + C_n \quad (1.11)$$

Phần tiếp theo trình bày quá trình xây dựng các công thức theo chiều nghịch, dựa trên thuật toán lan truyền ngược (backpropagation). Từ các công thức (1.1)–(1.10), có thể thấy C là một hàm phụ thuộc vào các trọng số và hệ số điều chỉnh ($w_{ij}^2, b_i^2, w_{ij}^3, b_i^3$). Do đó, đạo hàm của C theo các tham số này có thể được xác định.

Để tính đạo hàm của C theo các trọng số và hệ số điều chỉnh, cần áp dụng quy tắc đạo hàm của hàm hợp (chain rule), được phát biểu như sau: giả sử f là hàm của u và v , tức $f = f(u, v)$; u và v lần lượt là hàm của x , tức $u = u(x)$ và $v = v(x)$. Nếu các đạo hàm $f(u)$, $f(v)$, $u(x)$, $v(x)$ tồn tại thì:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial f}{\partial v} \frac{\partial v}{\partial x} \quad (1.12)$$

Xét trường hợp cần tính đạo hàm của C theo w_{11}^2 . Áp dụng quy tắc đạo hàm của hàm hợp, ta có:

$$\frac{\partial C}{\partial w_{11}^2} = \frac{\partial C_1}{\partial w_{11}^2} + \frac{\partial C_2}{\partial w_{11}^2} + \dots + \frac{\partial C_n}{\partial w_{11}^2} \quad (1.13)$$

$$\frac{\partial C_n}{\partial w_{11}^2} = \frac{\partial C_n}{\partial a_1^3[n]} \left(\frac{\partial a_1^3[n]}{\partial w_{11}^2} + \frac{\partial a_2^3[n]}{\partial w_{11}^2} \right) = (a_1^3[n] - t_1[n]) \left(\frac{\partial a_1^3[n]}{\partial w_{11}^2} + \frac{\partial a_2^3[n]}{\partial w_{11}^2} \right) \quad (1.14)$$

$$\frac{\partial a_1^3[n]}{\partial w_{11}^2} = \frac{\partial a_1^3[n]}{\partial z_1^3[n]} \frac{\partial z_1^3[n]}{\partial a_1^2[n]} \frac{\partial a_1^2[n]}{\partial z_1^2[n]} \frac{\partial z_1^2[n]}{\partial w_{11}^2} = \frac{\partial a_1^3[n]}{\partial z_1^3[n]} w_{11}^3 \frac{\partial a_1^2[n]}{\partial z_1^2[n]} k_1[n] \quad (1.15)$$

$$\frac{\partial a_2^3[n]}{\partial w_{11}^2} = \frac{\partial a_2^3[n]}{\partial z_2^3[n]} \frac{\partial z_2^3[n]}{\partial a_1^2[n]} \frac{\partial a_1^2[n]}{\partial z_1^2[n]} \frac{\partial z_1^2[n]}{\partial w_{11}^2} = \frac{\partial a_2^3[n]}{\partial z_2^3[n]} w_{12}^3 \frac{\partial a_1^2[n]}{\partial z_1^2[n]} k_1[n] \quad (1.16)$$

Từ các công thức (1.14)–(1.16), suy ra:

$$\frac{\partial C_n}{\partial w_{11}^2} = (a_1^3[n] - t_1[n]) \left(\frac{\partial a_1^3[n]}{\partial z_1^3[n]} w_{11}^3 + \frac{\partial a_2^3[n]}{\partial z_2^3[n]} w_{12}^3 \right) \frac{\partial a_1^2[n]}{\partial z_1^2[n]} k_1[n] \quad (1.17)$$

Kết hợp các công thức (1.13) và (1.17), có thể xác định được $\frac{\partial C}{\partial w_{11}^2}$.

Theo thuật toán gradient descent, giá trị mới của w_{11}^2 được cập nhật nhằm cực tiểu hóa C, như trình bày trong công thức (1.18):

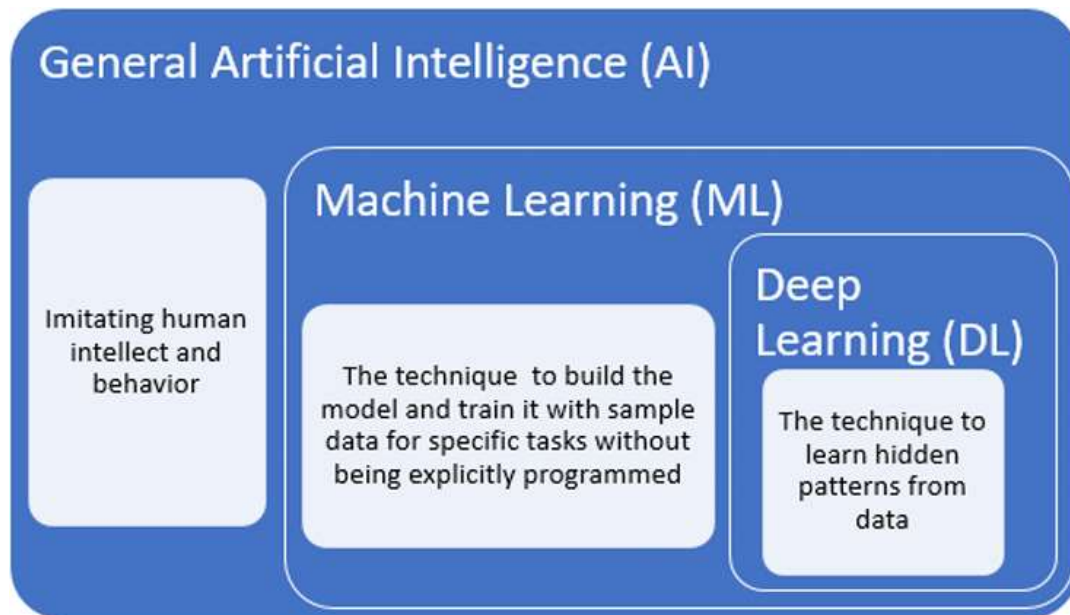
$$w_{11}^2(\text{new}) = w_{11}^2(\text{old}) - \eta \frac{\partial C}{\partial w_{11}^2} \quad (1.18)$$

Trong công thức (1.18), η là một hằng số dương, gọi là tốc độ học (learning rate). Sau khi cập nhật, quá trình đưa vector K vào mạng và thực hiện tính toán được lặp lại cho đến khi đầu ra của mạng đạt giá trị đủ gần với giá trị giám sát [1]. Mỗi lần mạng được huấn luyện qua toàn bộ tập hợp vector K của dữ liệu được gọi là một epoch [2].

2.2. Giới thiệu về Machine learning:

Machine Learning là một nhánh nhỏ hơn của AI, tập trung vào xây dựng các mô hình mạng neural và huấn luyện chúng với dữ liệu cho trước để máy tính có khả năng tự tối ưu và ứng phó với các bài toán mới [3].

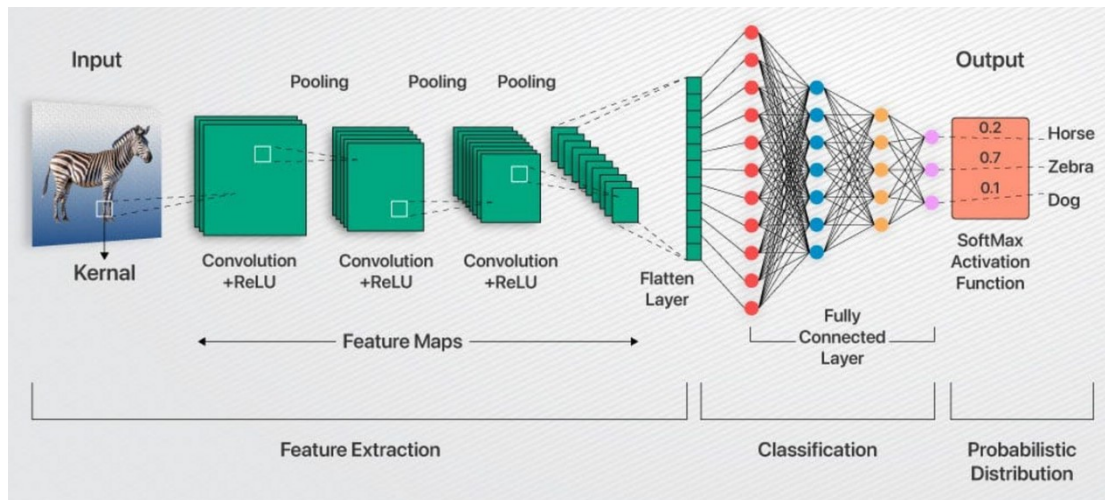
Deep Learning là công nghệ ML hiệu quả nhất hiện nay, nó sử dụng một mạng neural để thực hiện các phân tích và tính toán phức tạp trên các tập dữ liệu lớn [3], hình bên dưới mô tả về độ bao quát của các khái niệm: AI, ML, DL và cách để phân biệt chúng.



Hình 2.3: các phân loại trí tuệ nhân tạo [3]

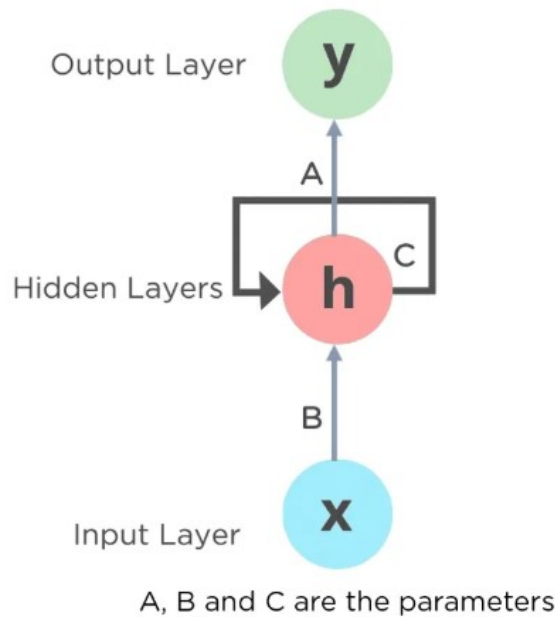
Deep Learning bao gồm rất nhiều cấu trúc mạng Neural khác nhau được tối ưu cho từng tác vụ cụ thể, một số kiểu mạng phổ biến nhất hiện nay bao gồm:

Mạng Nơ-ron Tích chập (CNN) là một trong những kiến trúc Deep Learning phổ biến và đạt hiệu quả cao nhất hiện nay, đặc biệt trong lĩnh vực xử lý ảnh. Nguyên lý hoạt động của CNN dựa trên phép tích chập ma trận, trong đó các bộ lọc (filter) trượt trên ma trận dữ liệu đầu vào nhằm trích xuất các đặc trưng không gian. Một cấu trúc CNN điển hình thường gồm bốn lớp chính: lớp Tích chập (Convolution) đảm nhận nhiệm vụ trích xuất đặc trưng; lớp Kích hoạt (Activation) bổ sung tính phi tuyến giúp mô hình học các biểu trưng phức tạp; lớp Gộp (Pooling) giảm chiều dữ liệu để tối ưu chi phí tính toán; và lớp Kết nối đầy đủ (Fully Connected - FC) đánh giá mối tương quan giữa các đặc trưng thu được để đưa ra kết quả phân loại cuối cùng.



Hình 2.1: Minh họa về hoạt động của một mạng CNN trong phân loại ảnh động vật

Bên cạnh CNN, Mạng Nơ-ron Hồi quy (RNN) là kiến trúc Deep Learning then chốt được thiết kế chuyên biệt cho các bài toán xử lý dữ liệu chuỗi và chuỗi thời gian nhờ đặc tính ghi nhớ các trạng thái trong quá khứ. Nguyên lý hoạt động của RNN dựa trên cơ chế vòng lặp phản hồi, trong đó trạng thái ẩn (hidden state) đóng vai trò như bộ nhớ nội bộ giúp lưu trữ và truyền tải thông tin ngữ cảnh qua từng bước thời gian. Một mô hình RNN điển hình thường được cấu thành từ ba thành phần chính: phần đầu vào (Input) tiếp nhận dữ liệu tuần tự tại mỗi bước thời gian; trạng thái ẩn (Hidden State) đảm nhận nhiệm vụ tổng hợp và duy trì thông tin từ các bước trước; và phần đầu ra (Output) đưa ra kết quả dự đoán dựa trên ngữ cảnh đã tích lũy; cùng với sự bổ sung của các biến thể cải tiến như LSTM hay GRU nhằm khắc phục hiệu quả hiện tượng triệt tiêu đạo hàm khi xử lý các chuỗi dữ liệu dài.



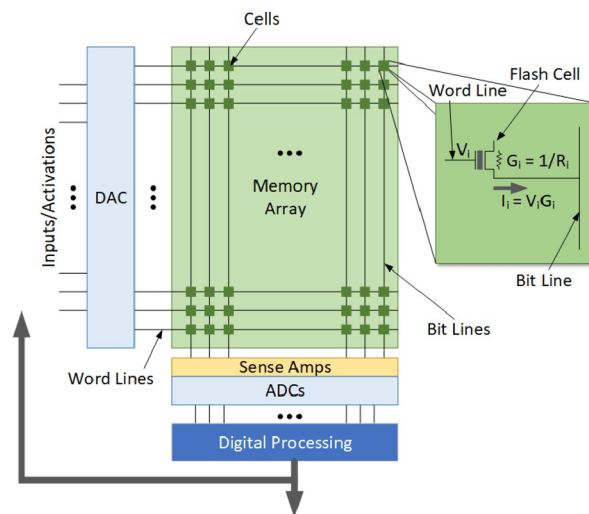
Hình 2.1: Cấu trúc cơ bản nhất của một mạng RNN

Ta thấy rằng, mặc dù mỗi kiến trúc mạng Neural được thiết kế theo các cách khác nhau để giải quyết một lớp bài toán đặc thù, điểm chung của chúng là đều được xây dựng xoay quanh phép nhân tích lũy ma trận MAC, đây chính là mục tiêu tối ưu chính của phần lớn thiết kế NPU hiện nay.

2.3. Khảo sát các nghiên cứu gần đây và đề xuất giải pháp - sử dụng song song analog CIM & MAC array:

Vấn đề lớn nhất mà kiến trúc Von Neumann truyền thống gặp phải khi xử lý các mạng neural là một lượng lớn năng lượng bị hao phí trên đường dữ liệu giữa bộ nhớ và các khối tính toán, cùng với đó là khả năng tính toán song song rất hạn chế dẫn đến độ trễ lớn trong quá trình xử lý. Vì thế, rất nhiều giải pháp thiết kế đã ra đời nhằm giải quyết vấn đề này, tiêu biểu nhất trong số đó là các NPU sử dụng mảng nhân tích lũy (MAC Array) và khối tính toán trong bộ nhớ (Compute-in-Memory - CIM). Các kiến trúc này ra đời dựa trên một ý tưởng chính: đưa khối tính toán đến gần nhất hoặc tích hợp vào bộ nhớ để tối thiểu hoá hao phí trong luân chuyển dữ liệu.

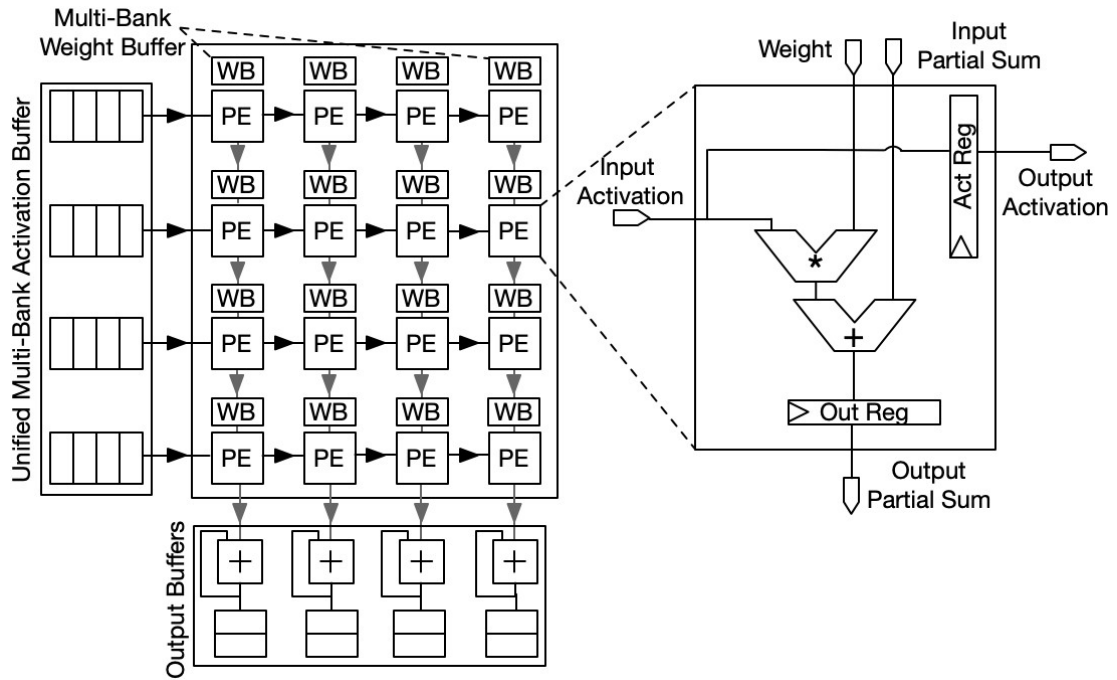
Có rất nhiều kiểu thiết kế CIM khác nhau nhưng chung quy lại chúng đều nhằm tối ưu phép nhân tích lũy ma trận MAC với các mảng bộ nhớ lưu trọng số được cải tiến để thực hiện tính toán ngay trên đó, hình dưới là một ví dụ về analog CIM:



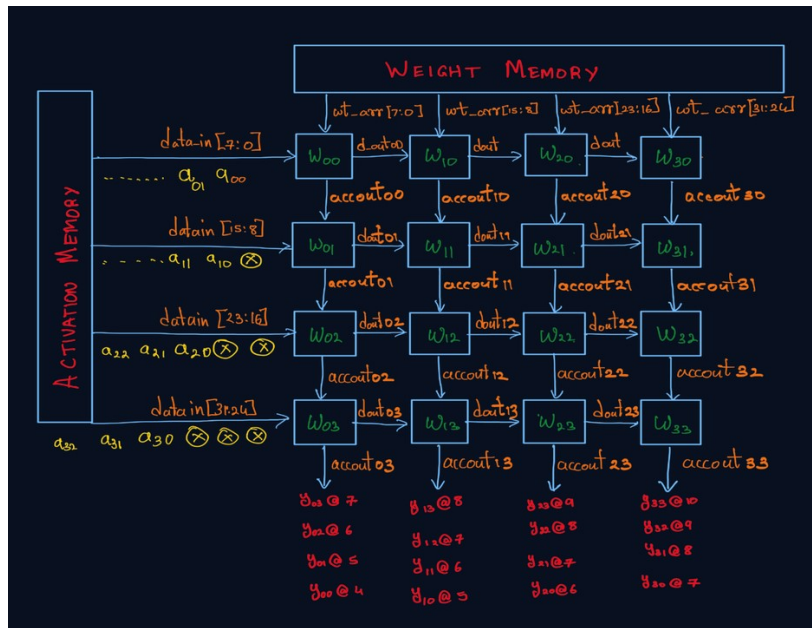
Hình 2.2: Analog Compute-in-memory

Khối CIM (ví dụ: dựa trên ReRAM hoặc SRAM) mang lại hiệu quả vượt trội về mặt năng lượng và tốc độ bằng cách triệt tiêu hiện tượng di chuyển dữ liệu (vượt qua “bức tường bộ nhớ” von Neumann) và cho phép tính toán song song cực lớn với độ phức tạp lý thuyết là $O(1)$. Tuy nhiên, CIM đối mặt với các thách thức nghiêm trọng về độ tin cậy vật lý như sai số ghi (write variations), trôi độ dẫn (conductance drift), nhiễu ngẫu nhiên, và nhạy cảm với biến động nhiệt độ/điện áp. Đặc biệt, trong các ứng dụng an toàn nghiêm ngặt (safety-critical) như xe tự hành hay điều khiển công nghiệp, những sai số nhỏ này có thể tích lũy, lan truyền qua các lớp và gây ra sự sụp đổ hoàn toàn về độ chính xác.

Để giải quyết vấn đề này, việc tích hợp một Digital MAC array làm khối dự phòng (fallback) là vô cùng thiết yếu. Khi khối CIM chính gặp lỗi hoạt động (do lỗi bit vật lý, lỗi điện áp thấp hoặc trôi độ dẫn theo thời gian) hoặc khi NPU cần xử lý các tác vụ yêu cầu độ chính xác cao và các thuật toán khó tối ưu hóa trên CIM (như các hàm phi tuyến phức tạp hay các lớp có tính chất memory-bound với độ tái sử dụng trọng số thấp), hệ thống có thể chuyển đổi linh hoạt sang khối Digital MAC thuần túy để bảo toàn tính toàn vẹn và độ tin cậy của phép tính.



Hình 2.3: Một mảng digital MAC processor theo cấu trúc systolic array



Hình 2.4: Systolic array bao gồm lưới phân tử MAC kích thước 4x4 và luồng dữ liệu của chúng.

* khảo sát các nghiên cứu trước đây:

Kiến trúc nghiên cứu	Công nghệ chính	Khối tính toán số (Digital MAC)	Khối CIM / PIM	Cơ chế hoạt động phối hợp	Hỗ trợ fallback động bảo vệ CIM
DianNao / DaDianNao	Digital ASIC	Có (Khối chức năng neural NFU)	Không	Chỉ sử dụng tính toán số học chính xác trên chip.	Không hỗ trợ.
Eyeriss / Eyeriss v2	Digital ASIC	Có (Mảng PE cấu trúc Row-Stationary)	Không	Tối ưu hóa việc tái sử dụng dữ liệu số (trọng số, kích hoạt, tổng riêng phần).	Không hỗ trợ.
PUMA	Lai Memristor - CMOS	Có (Khối chức năng vector - VFU)	Có (Memristor Crossbar MVMU)	Khối CIM đảm nhận phép nhân ma trận (MVM); các khối VFU số học xử lý phép toán phi tuyến (ReLU, Sigmoid) và hàm siêu việt.	Không hỗ trợ (không dùng khối số để làm dự phòng tính toán MVM thay thế cho CIM khi CIM lỗi).
NeuRRAM	Lai RRAM - CMOS	Không (chỉ có mạch CMOS ngoại vi điều khiển)	Có (RRAM Crossbar)	Mạch CMOS ngoại vi chỉ thực hiện truyền nhận dữ liệu, tích lũy analog và kích hoạt neuron.	Không hỗ trợ.
Hermes	Lai PCM - CMOS	Không (chỉ có mạch chuyển đổi ADC/DAC ngoại vi)	Có (PCM Crossbar)	Mạch ngoại vi tích hợp ADC/DAC và logic số để tính toán hàm kích hoạt và chuẩn hóa.	Không hỗ trợ.
DIANA (SoC)	Digital-Analog Hybrid	Có (Digital core)	Có (Analog CIM core)	Áp dụng tính toán lai chuyển đổi phân tầng theo từng lớp (layer-by-layer) của mô hình mạng neural.	Không hỗ trợ (khối số không hoạt động như một fallback động song song cho cùng một tác vụ tính toán của CIM khi xảy ra lỗi vật lý).

Tianjic	Lai Spiking (SNN) và Non-spiking (ANN)	Có (Logic số điều khiển và xử lý)	Có (Khối neuromorphic lai)	Linh hoạt lựa chọn SNN hoặc ANN cho từng lớp của mô hình để tối ưu hiệu năng và công suất.	Không hỗ trợ (chỉ chuyển đổi tinh cấu trúc lớp, không có cơ chế tự động fallback bảo vệ tin cậy).
Đề xuất của đề tài	Primary CIM + Digital MAC Fallback	Có (Digital MAC Array hỗ trợ đa độ chính xác)	Có (Primary CIM Block tính toán năng lượng thấp)	Khối CIM thực hiện hầu hết các phép toán MVM để tối ưu năng lượng. Khi phát hiện lỗi hoặc độ tin cậy CIM suy giảm (hoặc gặp các lớp đặc thù), hệ thống tự động chuyển vùng tính toán sang Digital MAC array để duy trì độ chính xác.	Có hỗ trợ (Cơ chế fallback giúp giải quyết triệt để bài toán tin cậy của CIM).

Tuy kiến trúc tích hợp trên cho thấy tiềm năng ứng dụng rất lớn, hầu hết các nghiên cứu hiện nay chỉ tập trung vào xây dựng một NPU sử dụng duy nhất CIM hoặc MAC array cho bộ tính toán trung tâm. Do đó nhóm tác giả đề xuất phương án thiết kế một bộ digital MAC array dùng chung dataflow và compiler với khối một khối CIM cho trước qua đó tạo tiền đề cho việc phát triển lên bộ NPU tích hợp CIM - MAC array sau này.

CHƯƠNG 3. THIẾT KẾ VÀ THỰC HIỆN PHẦN CỨNG

3.1. Bộ CIM tham chiếu:

Trong phần này ta sẽ tiến hành tham khảo một khối CIM sẵn có để dựa vào đó đặt ra spect về dataflow và các thông số khác cho khối MAC array fallback sẽ thiết kế. Sau khi xem xét các nghiên cứu khác nhau, nhóm tác giả quyết định lựa chọn nghiên cứu: “Input Sparsity-Aware Computing-In-Memory with Bidirectional Conversion-Skipable Analog-to-Digital Converter” của Choongseok Song và cộng sự để làm nguồn tham khảo chính.

3.1.1. Cơ sở lựa chọn kiến trúc CIM của Song và cộng sự (2026) làm hệ quy chiếu

Nhằm xây dựng giải pháp thiết kế cho mảng nhân – cộng dồn (Multiply-Accumulate – MAC) kỹ thuật số dự phòng (Digital NPU Fallback, ký hiệu A2), đề tài lựa chọn kiến trúc điện toán trong bộ nhớ dạng tương tự trên nền SRAM (SRAM-based Analog Computing-in-Memory – CIM) do nhóm tác giả Choongseok Song, Yonguk Sim và Doo Seok Jeong công bố năm 2026 làm nền tảng tham chiếu chủ đạo. Việc lựa chọn này được luận giải trên cơ sở ba luận điểm kỹ thuật và thực tiễn cốt lõi, trình bày lần lượt dưới đây.

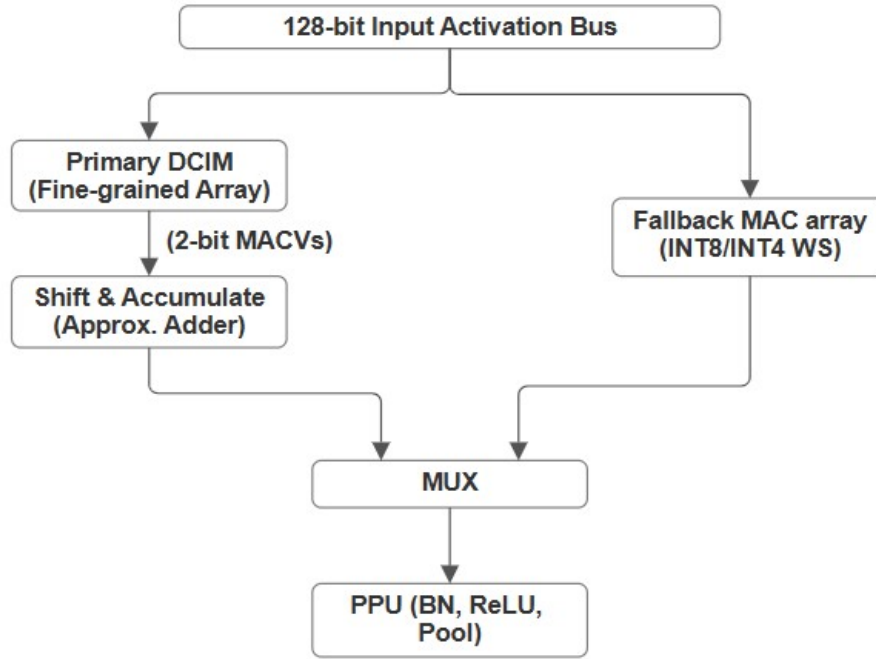
Thứ nhất, về sự tương thích giữa luồng dữ liệu (dataflow) và trình biên dịch (compiler). Kiến trúc CIM tham chiếu vận hành trên nguyên lý weight-stationary (WS), theo đó trọng số được nạp và lưu giữ cố định tại các tế bào nhớ, trong khi dữ liệu kích hoạt đầu vào (Input Activation – IA) được đưa vào mảng tính toán theo cơ chế bit-serial, tức là truyền tuần tự từng bit từ MSB đến LSB. Cấu trúc luồng dữ liệu này tương đồng hoàn toàn với mảng nhân-cộng dồn kỹ thuật số dự phòng dạng nối tiếp (Serial-MAC – SMAC) được đề xuất trong đề tài. Nhờ đặc tính tương thích này, hệ thống dự phòng có thể sử dụng chung toàn bộ trình biên dịch cũng như bộ sinh địa chỉ nạp dữ liệu với khối CIM chính; khi phát sinh lỗi vật lý trên nhánh CIM, bộ điều khiển chỉ cần chuyển hướng (bypass) luồng dữ liệu kích hoạt sang mảng MAC số dự phòng mà không đòi hỏi biên dịch lại mô hình mạng nơ-ron.

Thứ hai, về tính khả thi trong quy trình thực thi vật lý (RTL-to-GDS). Khác với các kiến trúc CIM dựa trên công nghệ bộ nhớ phi biến đổi mới nổi (như ReRAM hoặc PCM), vốn đòi hỏi các công đoạn xử lý phi tiêu chuẩn trong chế tạo, kiến trúc tham chiếu của Song và cộng sự (2026) được xây dựng trên nền tế bào 8T-SRAM tiêu chuẩn, hoàn toàn tương thích với quy trình thiết kế và chế tạo logic CMOS thông thường. Nhờ đó, khối CIM chính và mảng MAC kỹ thuật số dự phòng của đề tài có thể được tích hợp trên cùng một die chip, sử dụng chung một luồng tổng hợp logic và bố trí vật lý (tổng hợp logic, phân tích thời gian tĩnh – STA, và bố trí – đi dây – P&R) trên cùng một thư viện công nghệ CMOS tiêu chuẩn, cụ thể là tiến trình 45-nm CMOS được lựa chọn làm mục tiêu của đề tài.

Thứ ba, về khả năng giải quyết triệt để bài toán độ tin cậy trong các tác vụ nhạy cảm (ECG và KWS). Mặc dù khối lõi CIM dạng tương tự đạt hiệu suất năng lượng cao nhờ tối ưu hóa chu kỳ chuyển đổi tương tự – số, các phép toán tương tự luôn tiềm ẩn nguy cơ sai lệch do các yếu tố biến thiên vật lý không thể kiểm soát hoàn toàn, như nhiễu ngẫu nhiên, hiện tượng trôi độ dẫn và biến động theo quá trình – điện áp – nhiệt độ (PVT). Trong các tác vụ y tế biên có yêu cầu độ chính xác cao, như phân loại nhịp điện tâm đồ (ECG) hoặc nhận diện từ khóa giọng nói (KWS), các sai lệch tương tự dù nhỏ cũng có nguy cơ tích lũy và gây suy giảm đáng kể độ chính xác của mô hình. Việc thiết kế một mảng MAC số dự phòng có tính tất định (deterministic), đóng vai trò như một tuyến dự phòng an toàn cho hệ thống trên chip (SoC) sử dụng CIM tham chiếu, do đó là cần thiết nhằm bảo đảm độ chính xác toán học tuyệt đối trong trường hợp điều kiện vận hành hoặc mức độ lỗi vật lý của mảng CIM vượt quá ngưỡng cho phép.

3.1.2. Sơ đồ khối và nguyên lý hoạt động của bộ CIM tham chiếu

Kiến trúc CIM tham chiếu được xây dựng trên nguyên lý tính toán tương tự kết hợp với logic nhận biết độ thưa (sparsity-aware) nhằm cắt giảm các thao tác tính toán dư thừa. Sơ đồ khối chức năng của hệ thống bao gồm bốn thành phần chính, được minh họa trong hình dưới và trình bày chi tiết trong các mục tiếp theo.



Hình 3.1: Sơ đồ khối chức năng của bộ NPU dùng CIM tích hợp mảng MAC dự phòng

Thành phần nền tảng của CIM macro tham chiếu là mảng tế bào nhớ tương tự 8T-SRAM, đóng vai trò lõi lưu trữ với cấu trúc 256×8 tế bào nhớ. Khác với tế bào 6T-SRAM truyền thống chỉ đảm nhiệm chức năng lưu trữ nhị phân thuần túy, tế bào 8T-SRAM được bổ sung hai transistor vượt kênh (pass transistor) đóng vai trò một bộ nhân nội tại ngay tại ô nhớ. Cổng của hai transistor này được điều khiển đồng thời bởi bit dữ liệu lưu trữ trong ô nhớ, đóng vai trò trọng số ($d[i]$), và tín hiệu đầu vào (IN) do bộ kích hoạt cung cấp; khi cả hai tín hiệu đồng thời đạt mức logic “1”, bộ nhân được kích hoạt và cho phép dòng điện phóng qua đường bitline đọc (Read Bitline – RBL), vốn đã được nạp sẵn điện áp tham chiếu V_{ref} . Nhờ cơ chế này, phép nhân – cộng dồn tương tự (analog Multiply-Accumulate – MAC) được thực hiện tức thời ngay tại lớp vật lý của bộ nhớ, tuân theo định luật Kirchhoff về dòng điện, thông qua việc cộng gộp một cách tự nhiên các dòng điện phóng từ toàn bộ các ô nhớ có chung đường RBL – qua đó loại bỏ nhu cầu di chuyển dữ liệu trọng số ra khỏi mảng nhớ để thực hiện phép tính, vốn là nguyên nhân chính gây tổn hao năng lượng trong các kiến trúc tính toán truyền thống.

Việc cấp phát dữ liệu kích hoạt cho mảng tính toán được đảm nhiệm bởi bộ kích hoạt IA (Input Activation Driver), có chức năng tiếp nhận dữ liệu kích hoạt 8-bit, lựa chọn và truyền tuần tự (bit-serial) từng bit đến các hàng tính toán tương ứng. Đáng chú ý, bộ kích hoạt IA được tích hợp thêm một mạch đếm số lượng bit “1” (Population Counter – Pop-count), có nhiệm vụ xác định đồng thời số lượng bit kích hoạt mang giá trị “1” trên 32 hàng song song trong mỗi chu kỳ xung nhịp. Giá trị tổng số bit hoạt động, ký hiệu IN_sum, được truyền trực tiếp đến bộ điều khiển trung tâm và bộ chuyển đổi tương tự – số, đóng vai trò cơ sở định lượng để xác lập động chế độ nhảy bước phù hợp cho từng chu kỳ chuyển đổi – đây chính là điểm mấu chốt cho phép CIM macro khai thác đặc tính thưa (sparsity) của dữ liệu đầu vào ở mức phần tử (element-level).

Cải tiến trọng yếu nhất trong kiến trúc CIM macro tham chiếu nằm ở bộ chuyển đổi tương tự – số nhảy bước hai hướng (Bidirectional Conversion-Skippable ADC – BCS-ADC), được phát triển trên nền cấu trúc SAR-ADC 5-bit truyền thống với việc bổ sung một khối logic nhảy bước (BCS logic). Khối logic này cho phép BCS-ADC tự động điều chỉnh động độ phân giải bit của quá trình chuyển đổi theo hai hướng đối lập nhằm tối ưu đồng thời độ trễ và năng lượng tiêu thụ. Theo hướng thứ nhất, gọi là nhảy bước phía bit cao (MSB-side skipping – Mskip), BCS-ADC khai thác một bất đẳng thức vật lý cho thấy giá trị MAC thực tế (MACV) luôn nhỏ hơn hoặc bằng số lượng đầu vào tích cực IN_sum; theo đó, khi IN_sum càng nhỏ thì các bit trọng số lớn (MSB) của kết quả càng có xác suất chắc chắn bằng 0. Trên cơ sở này, giá trị IN_sum được sử dụng để phân loại động chu kỳ chuyển đổi thành năm chế độ, ký hiệu Mskip0 đến Mskip4, tương ứng với các khoảng giá trị IN_sum giảm dần – từ toàn dải 16–32 đối với Mskip0 (không nhảy bước) cho đến giá trị bằng 1 đối với Mskip4; tại mỗi chế độ, BCS-ADC chủ động loại bỏ các chu kỳ chuyển đổi MSB tương ứng, giảm từ năm chu kỳ chuẩn xuống còn tối thiểu một chu kỳ, đồng thời triệt tiêu hoàn toàn hoạt động chuyển mạch điện dung của các tụ điện có giá trị lớn nhất trong mảng CDAC (như tụ C_(b-1)) đảm nhiệm việc chuyển đổi MSB) – vốn là nguồn tiêu thụ năng lượng chủ yếu của bộ chuyển đổi. Theo

hướng thứ hai, gọi là nhảy bước phía bit thấp (LSB-side skipping – Lskip), BCS-ADC được phép chủ động kết thúc sớm quá trình chuyển đổi trước khi xử lý đến các chu kỳ ứng với bit trọng số thấp, tương đương về mặt toán học với phép làm tròn xuống giá trị MACV; cơ chế này giúp tiết kiệm thêm thời gian và năng lượng chuyển đổi, song có thể đánh đổi bằng một mức suy giảm độ chính xác nhất định nếu được áp dụng không chọn lọc. Ở giới hạn cực đoan của nhảy bước phía MSB, khi mạch Pop-count phát hiện toàn bộ 32 kích hoạt đầu vào trong một chu kỳ đều bằng 0 (tức $IN_sum = 0$), BCS-ADC kích hoạt chế độ bỏ qua ở cấp khối (Block-level skipping), theo đó toàn bộ chu kỳ đọc được ngắt hoàn toàn và giá trị 0 được xuất trực tiếp ra ngoài mà không cần thực hiện bất kỳ phép chuyển đổi nào.

Khép lại chuỗi xử lý, kết quả kỹ thuật số thu được song song từ tám bộ BCS-ADC được đưa vào khối dịch và tích lũy (Shift-and-Add – SnA) để hoàn tất phép tính. Khối SnA thực hiện dịch căn lề (bit alignment) các giá trị này theo đúng trọng số nhị phân của digit kích hoạt đang được nạp trong chu kỳ bit-serial hiện hành, đồng thời tích lũy lặp qua toàn bộ tám chu kỳ nạp kích hoạt nhằm tái tạo chính xác kết quả tích chập INT8 hoàn chỉnh, khép lại toàn bộ chuỗi xử lý của một phép nhân – cộng dồn trong CIM macro tham chiếu.

3.1.3. Các thông số kỹ thuật cốt lõi của CIM macro tham chiếu

Bảng bên dưới tổng hợp các thông số kỹ thuật, kích thước phần cứng vật lý và các chỉ số hiệu năng thực nghiệm của CIM macro tham chiếu, được thiết kế trên tiến trình công nghệ 45-nm CMOS tiêu chuẩn.

Bảng 1.1. Thông số kỹ thuật và hiệu năng thực nghiệm của CIM macro tham chiếu

Tham số tham chiếu	Đặc tả CIM macro tham chiếu	Yêu cầu suy ra cho mảng MAC dự phòng (A2 – Digital NPU Fallback)
Tiến trình chế tạo & khả năng thực thi vật lý	45-nm CMOS chuẩn (Cadence GPDK045), tế bào 8T-SRAM tương thích logic CMOS thông thường	Phải hiện thực trên cùng thư viện CMOS chuẩn, đưa được xuống gate-level và physical (tổng hợp logic, STA, P&R) để tích hợp chung die với khối CIM
Điện áp nguồn (V_{DD})	1,0 V	Vận hành tương thích ở cùng mức điện áp nguồn với khối CIM chính trong cùng SoC

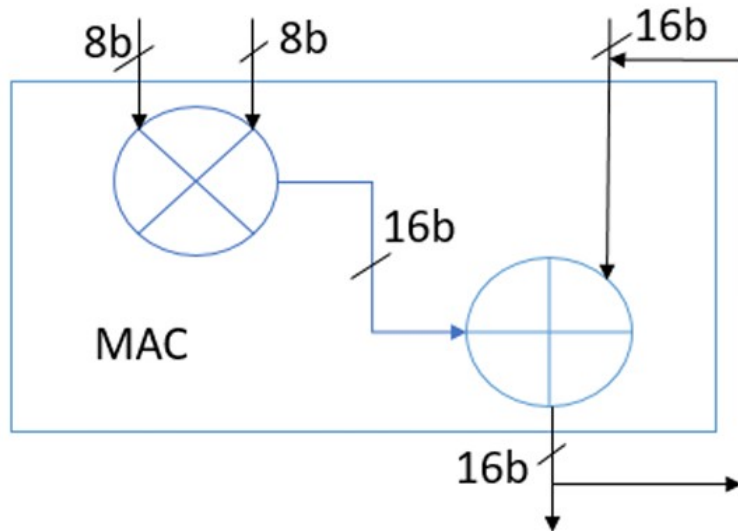
Tham số tham chiếu	Đặc tả CIM macro tham chiếu	Yêu cầu suy ra cho mảng MAC dự phòng (A2 – Digital NPU Fallback)
Tần số xung nhịp	100 MHz	Đáp ứng thông lượng tối thiểu trong đương ở cùng tần số, tránh nút thắt cổ chai khi chuyển sang chế độ dự phòng
Độ phân giải dữ liệu	INT8 kích hoạt (bit-serial) / INT8 trọng số	Hỗ trợ INT8/INT4, dùng chung định dạng dữ liệu để tương thích trực tiếp tại giao diện với khối CIM
Luồng dữ liệu (Dataflow)	Weight-stationary (WS), kích hoạt nạp theo cơ chế bit-serial	Dùng chung kiểu weight-stationary và chung trình biên dịch / bộ sinh địa chỉ với CIM để bypass tức thời, không cần biên dịch lại mô hình

3.2. Thông số kĩ thuật:

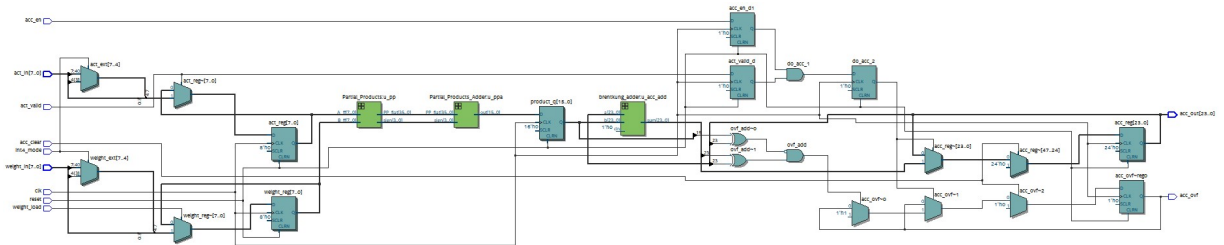
Dựa vào bộ CIM tham chiếu bên trên và mục tiêu nhắm đến các hệ thống AI biên quy mô nhỏ yêu cầu tiết kiệm năng lượng và độ chính xác cao, ta thấy rằng thiết kế bộ MAC array cần đạt được các specs sau đây.

Thông số	Spect	Giải thích
Tần số xung nhịp (f _{clk})	> 100MHz	
Công nghệ	45nm	Đồng bộ thiết kế
Supply Voltage	1.0V	Ổn định tốt và đồng bộ thiết kế
Accuracy	~100%	Fallback tốt cho các ứng dụng yêu cầu độ chính xác cao như ECG

3.3. Thiết kế khối MAC unit (Processing Element):



Hình 3.2: Sơ đồ nguyên lý bộ MAC unit



Hình 3.3: tổng quan module PE sau khi synthesize

Ta thấy rằng để thiết kế một khối MAC unit, trước hết ta cần phải thiết kế bộ nhân và cộng vì chúng đóng vai trò cốt lõi trong một cấu trúc MAC.

3.3.1. Bộ nhân:

Sau khi xem xét nhiều cách thiết kế bộ nhân khác nhau, nhóm tác giả nhận thấy bộ nhân modified booth sử dụng kiểu mã hoá radix-4 phù hợp nhất với các mục tiêu và specs đã đề ra.

Multiplier	Total Area (μm^2)	Total Power (mW)	Latency (ns)	Average Fan-out
Array ¹	1053.996	0.749	1.097	1.8
Modified Booth	531.294	0.476	0.964	1.8
Braun	783.200	0.733	0.990	1.8
Baugh-Wooley	584.909	0.498	0.964	1.4
Vedic-RCA	839.146	0.679	0.984	1.6
Wallace	768.623	0.537	0.585	2.2

Bảng 3.1: So sánh về area, power, latency và fan-out trung bình của các kiến trúc bộ nhân tại $CLK = 960\text{MHz}$ [3]

Bảng dữ liệu trên so sánh các kiến trúc multiplier dựa trên hai chỉ số cốt lõi: total area và total power consumption. Quá trình tổng hợp được cấu hình tại tần số xung nhịp mục tiêu 1 GHz, đồng thời thông số average fan-out được trình bày độc lập. Về mặt kỹ thuật, fan-out của một gate output phản ánh số lượng inputs mà đầu ra đó có khả năng điều khiển; chỉ số này quyết định trực tiếp đến hiệu quả định tuyến (routing) cũng như khả năng thỏa mãn các ràng buộc thời gian của toàn bộ thiết kế.

Kết quả thực nghiệm cho thấy Modified Booth multiplier đạt độ tối ưu cao nhất khi sở hữu area footprint nhỏ nhất cùng mức power usage thấp nhất. Trong khi đó, kiến trúc Baugh-Wooley xếp thứ hai về khả năng tối ưu area và power consumption, nhưng lại gặp hạn chế do sở hữu chỉ số fan-out thấp nhất [3]. Vì vậy nhóm quyết định chọn modified booth multiplier cho thiết kế.

Quá trình hoạt động của thuật toán Radix-4 Modified Booth bắt đầu bằng việc quét đồng thời từng cụm ba bit chồng lấp trên thừa số (multiplier) Y , thay vì xét lần lượt từng bit như phương pháp nhân cổ điển. Mỗi cụm bao gồm ba vị trí liên kề ($y_{2i+1}, y_{2i}, y_{2i-1}$), trong đó bit thấp nhất của cụm đầu tiên được khởi tạo bằng 0 ($y_{-1} = 0$) để đảm bảo phép quét bắt đầu đúng vị trí. Tại mỗi bước quét, mạch giải mã sẽ đọc giá trị ba bit này và chuyển đổi thành một hệ số điều khiển $k_i \in \{-2, -1, 0, 1, 2\}$, theo công thức:

$$k_i = -2y_{2i+1} + y_{2i} + y_{2i-1}$$

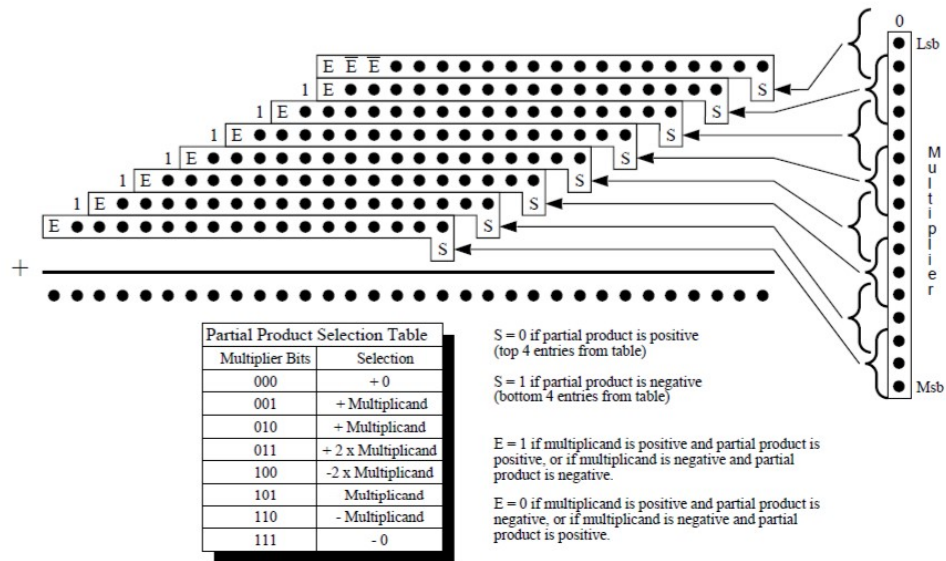
Multiplier Bits Block			Recoded 1-bit pair		2 bit booth	
i+1	i	i-1	i+1	i	Multiplier Value	Partial Product
0	0	0	0	0	0	Mx0
0	0	1	0	1	1	Mx1
0	1	0	1	-1	1	Mx1
0	1	0	1	0	2	Mx2
1	0	0	-1	0	-2	Mx-2
1	0	1	-1	1	-1	Mx-1
1	1	0	0	-1	-1	Mx-1
1	1	0	0	0	0	Mx0

Hình 3.4: Bảng mã hoá radix-4 cho bộ nhân Booth

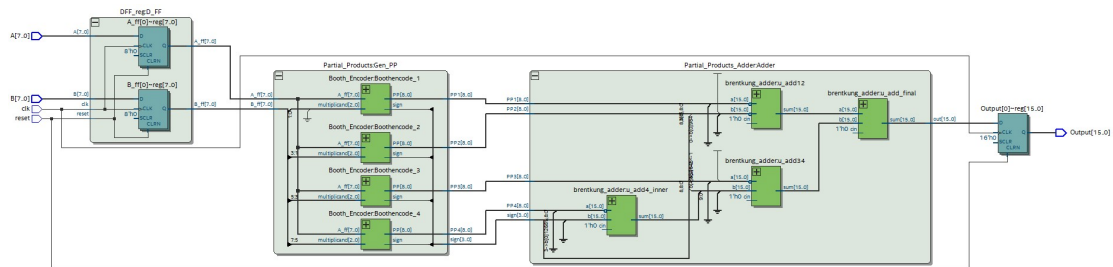
Hệ số k_i thu được sau bước giải mã sẽ điều khiển trực tiếp việc tạo ra sản phẩm trung gian thứ i (partial product) PP_i , thông qua phép nhân giữa k_i với số bị nhân (multiplicand) X :

$$PP_i = k_i \times X$$

Trong thực thi phần cứng, mạch tạo PP_i không thực hiện phép nhân theo nghĩa thông thường mà vận hành theo cơ chế chọn kênh: khi $k_i = \pm 1$, mạch MUX chọn trực tiếp giá trị X ; khi $k_i = \pm 2$, X được dịch trái một bit trước khi đưa qua MUX; và khi k_i mang dấu âm, cổng XOR thực hiện đảo bit toàn bộ giá trị đã chọn, cộng thêm 1 ở tầng cộng dồn để hoàn tất biểu diễn bù hai. Sau khi toàn bộ $[N/2]$ sản phẩm trung gian được tạo ra theo cơ chế trên, các PP này được đưa vào cây cộng dồn (reduction tree), điển hình là CSA Tree (Carry-Save Adder Tree), để cộng dồn song song qua nhiều tầng cho đến khi thu được hai giá trị cuối cùng, sau đó được cộng lại bằng một bộ cộng nhanh để tạo ra kết quả tích cuối cùng.

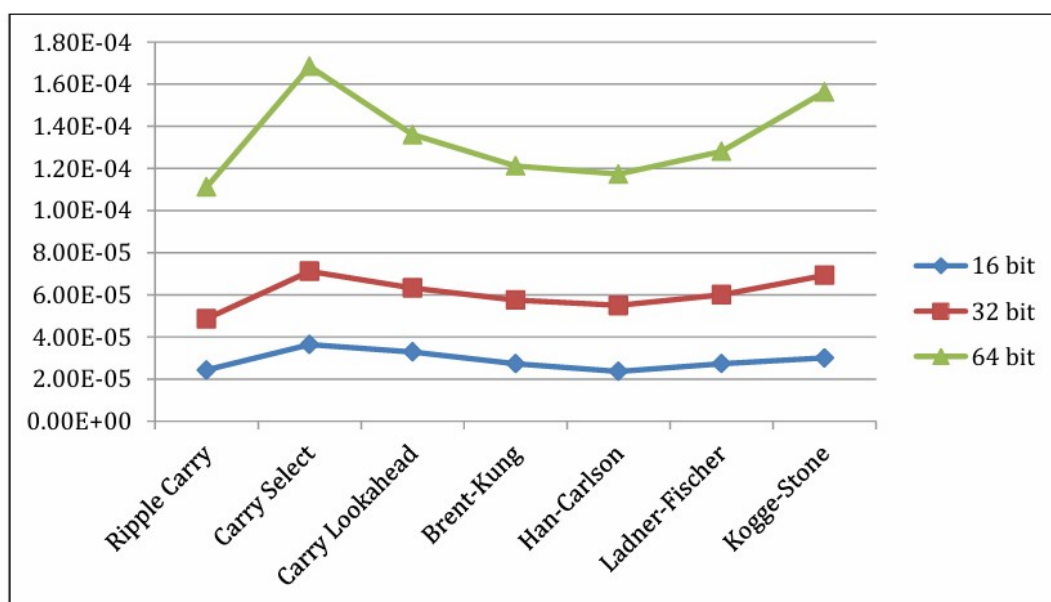


Hình 3.5: Sơ đồ chấm Radix-4 minh họa quá trình căn chỉnh tích riêng phần và mở rộng dấu của thuật toán booth multiply [4].

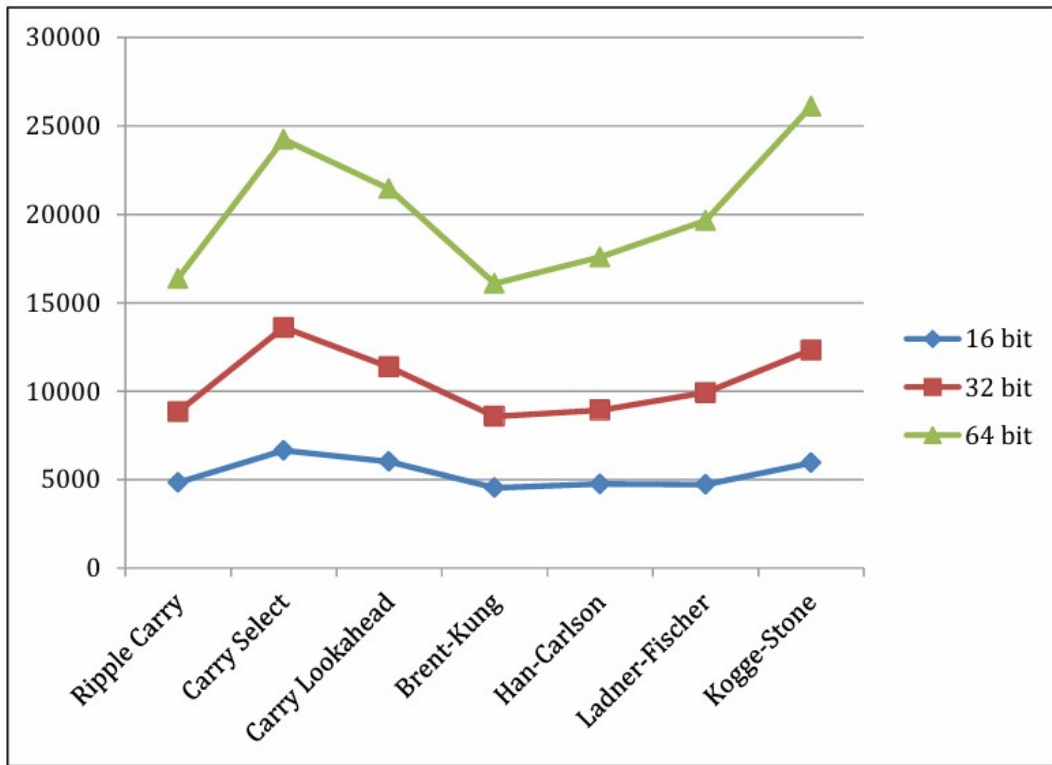


Hình 3.6: Bộ nhân booth cải tiến sau khi synthesize

3.3.2. Bộ cộng:



Hình 3.7: so sánh về power giữa các kiểu bộ cộng [5]



Hình 3.8: so sánh về area giữa các kiểu bộ cộng [5]

	16 Bit	32 Bit	64 Bit
Ripple Carry	2.2987	4.2099	9.545
Carry Select	1.6825	2.2043	4.1861
Carry Lookahead	1.5346	1.9725	2.202
Brent-Kung	0.7493	1.0528	1.3653
Ladner-Fischer	0.6671	0.8042	1.0805
Han-Carlson	0.664	0.8546	1.047
Kogge-Stone	0.6572	0.7803	0.971

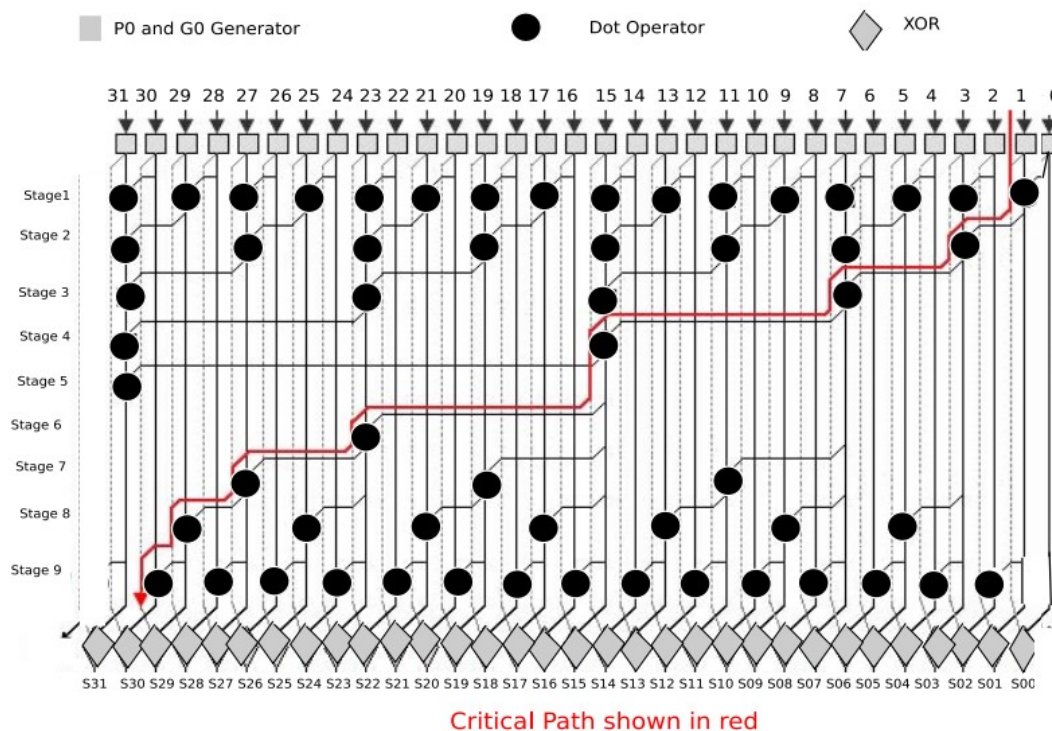
Hình 3.9: so sánh về post APR delay giữa các kiểu bộ cộng [5]

Sau khi xem xét các so sánh tương quan về thông số giữa các kiến trúc bộ cộng, và đối chiếu với nhu cầu cần một bộ cộng không cần quá nhanh

nhưng tiết kiệm area và năng lượng, ta nhận thấy Brent-Kung đáp ứng tốt nhu cầu này mà không đánh đổi quá nhiều về độ trễ.

Bộ cộng Brent–Kung là kiến trúc bộ cộng song song dạng cây tiền tố (parallel-prefix adder), thuộc cùng họ với Kogge–Stone, Han–Carlson và Ladner–Fischer. Điểm khác biệt của Brent–Kung so với các cấu trúc còn lại nằm ở việc kiến trúc này được xây dựng theo hai pha đối xứng: một pha thu gọn tiền (forward reduction) và một pha nội suy lùi (backward interpolation), nhằm tối thiểu hóa số lượng phần tử tính toán (dot operator) trong khi vẫn giữ độ trễ logic ở mức chấp nhận được. Đây là lý do Brent–Kung thường được lựa chọn cho các khối cộng trong datapath của các bộ gia tốc phần cứng có ràng buộc chặt về diện tích và công suất, ví dụ thực tế nhất chính là mảng MAC trong kiến trúc NPU hướng tới ứng dụng TinyML mà ta đang thực hiện.

Hình 3.10 mô tả cấu trúc bộ cộng Brent–Kung 32-bit, gồm 32 cột (tương ứng bit 0 đến bit 31), 9 tầng dot-operator (Stage 1 đến Stage 9), và một hàng cổng XOR ở đáy để tạo tổng cuối cùng S0–S31. Số tầng $9 = 2 \cdot \log_2(32) - 1 = 2 \cdot 5 - 1$ chính là đặc trưng độ sâu logic của Brent–Kung cho từ $n = 32$ bit.



Hình 3.11: sơ đồ khối của một bộ cộng Brent-Kung 32-bit

Với hai toán hạng A và B, tại mỗi bit i , hệ thống định nghĩa hai tín hiệu cơ sở, được tạo ra ở hàng "P0 and G0 Generator" trên cùng của hình:

$$P_i = A_i \oplus B_i \quad (\text{propagate - bit nhớ lan truyền})$$

$$G_i = A_i \cdot B_i \quad (\text{generate - bit nhớ tự sinh})$$

Toán tử Dot “•” (ký hiệu hình tròn đen trong sơ đồ) là một toán tử kết hợp (associative operator), dùng để gộp một cặp tín hiệu (G, P) của nhóm bit $[i:k]$ với nhóm bit $[k-1:j]$ liền trước thành tín hiệu (G, P) của nhóm gộp $[i:j]$:

$$(G, P)_{i:j} = (G, P)_{i:k} \cdot (G, P)_{k-1:j}$$

$$G_{i:j} = G_{i:k} + (P_{i:k} \cdot G_{k-1:j})$$

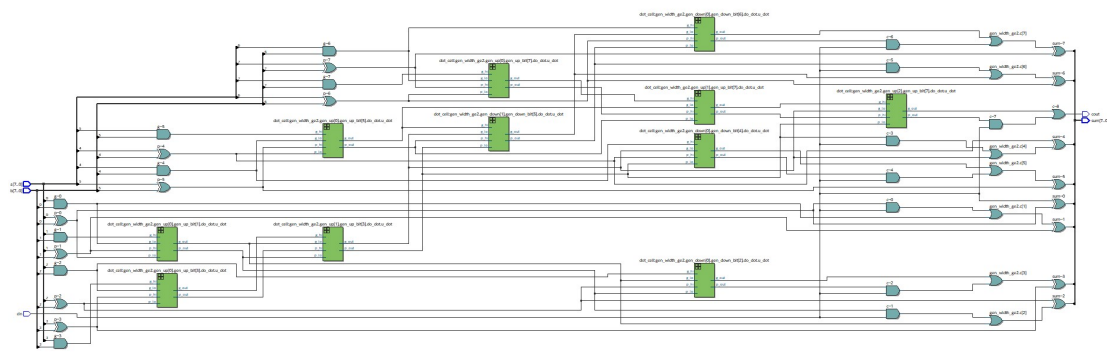
$$P_{i:j} = P_{i:k} \cdot P_{k-1:j}$$

Vì toán tử dot “•” có tính kết hợp, các dot-operator có thể được sắp xếp lại thành nhiều cấu trúc cây khác nhau (Kogge–Stone, Brent–Kung, Han–Carlson...) mà vẫn cho cùng kết quả toán học, chỉ khác nhau về số lượng phần

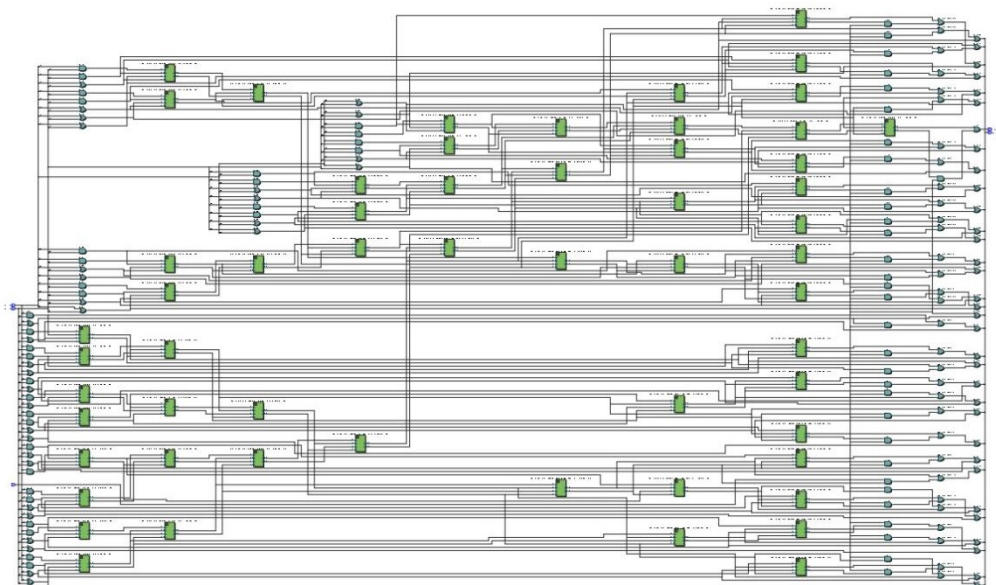
tử và độ sâu logic. Sau khi có carry-in tổng hợp $C_i = G_{i:0}$ (giả sử $C_{-1} = 0$), tổng từng bit được tính bằng cổng XOR ở hàng cuối cùng của sơ đồ:

$$S_i = P_i \oplus C_{i-1}$$

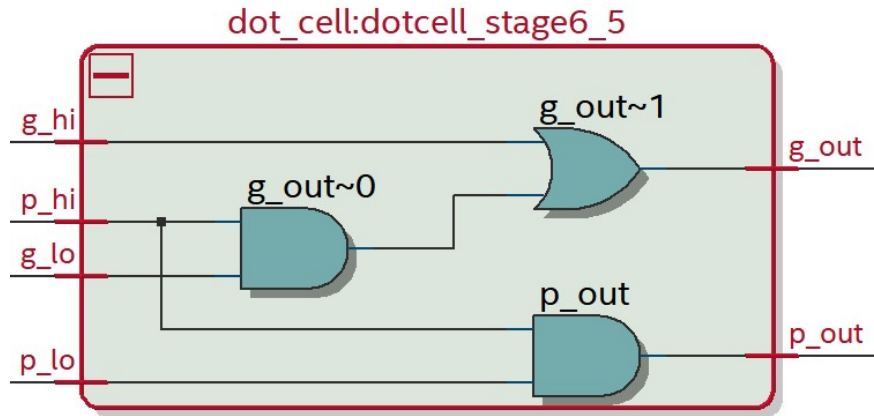
Tóm lại, với mảng MAC INT8/INT4 weight-stationary, phần cộng tích lũy (accumulator) là thành phần lặp lại nhiều lần và ảnh hưởng trực tiếp đến diện tích cũng như công suất tổng thể của mảng. Nhờ độ phức tạp phần cứng $O(n)$ thay vì $O(n \cdot \log n)$ như các parallel-prefix adder khác, Brent-Kung là lựa chọn cân bằng hợp lý giữa diện tích/wiring congestion, power và độ trễ, phù hợp nhất với mục tiêu ta cần hướng đến.



Hình 3.12: bộ cộng 8-bit Brent-Kung adder sau khi synthesize



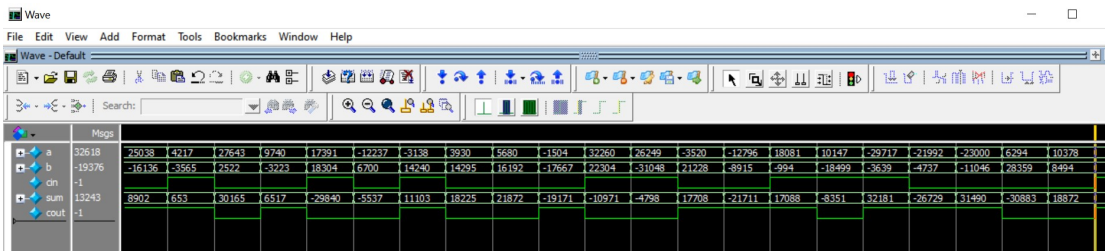
Hình 3.13: bộ cộng 32-bit Brent-Kung adder sau khi synthesize



Hình 3.14: khối dot_cell xử lý phép tính dot

3.4. Kiểm chứng thiết kế:

3.4.1. Khối PE:



Hình 3.16: Dạng sóng kiểm chứng bộ cộng Brent-Kung



Hình 3.17: Dạng sóng kiểm chứng thiết kế module MAC unit (PE)

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Qua kì thực tập, đề tài bước đầu cơ bản đã hoàn thành xong quá trình xây dựng cơ sở lý thuyết, khảo sát các nghiên cứu hiện có và thành công xây dựng khối MAC-unit sử dụng modified booth multiplier và Brent-Kung adder.

Trong giai đoạn tiếp theo tác giả sẽ tiến đến xây dựng khối MAC array hỗ trợ INT8/INT4 và weigh stationary, verify và bắt đầu quá trình chạy thử nghiệm thiết kế trên FPGA, đánh giá hiệu năng và so sánh với các nghiên cứu khác. Mục tiêu cuối cùng của đồ án sẽ là xây dựng một IP được triển khai đến mức GDSII.

CHƯƠNG 5. TÀI LIỆU THAM KHẢO

- [1] "21st LSI Design Contests-in Okinawa Design Specification - 3," LSI Design Contest, 2018. [Online]. Available: https://www.lsi-contest.com/2018/shiyou_3e.html. [Accessed 17 8 2026].
- [2] V. H. Tiệp, Machine learning cơ bản, <https://machinelearningcoban.com/>, 2018, March 27.
- [3] A. Samaneh, Neural Network Accelerator Design for System on Chip, Tampere University, 2022.
- [4] R. Sharma, 2026. [Online]. Available: <https://github.com/ramansh007/Signed-Booth-Multiplier/>. [Accessed 19 8 2026].
- [5] S. Dalmiya, "A Comparative Study of Adders," The University of Texas at Austin , December 2015.
- [6] Y. Qin, J. Zheng, Y. Shi, X. S. Hu, W. Wen and Z. Yan, "When Small Variations Become Big Failures: Reliability Challenges in Compute-in-Memory Neural Accelerators," Mar 2026.
- [7] R. a. W. Z. a. L. Q. a. G. B. a. H. Z. a. G. T. a. D. S. a. Z. J. a. Q. Q. a. W. D. a. o. Yu, "A full-stack memristor-based computation-in-memory system with software-hardware co-development," Nature Publishing Group UK London, 2025.
- [8] B. Liu, Q. Wei, Y. Zhang, X. Xu, Z. Zou, X. Huang, X. Si and H. Cai, "FDCA: Fine-grained Digital-CIM based CNN Accelerator with," *DAC'24*, 2024.